

The command reference

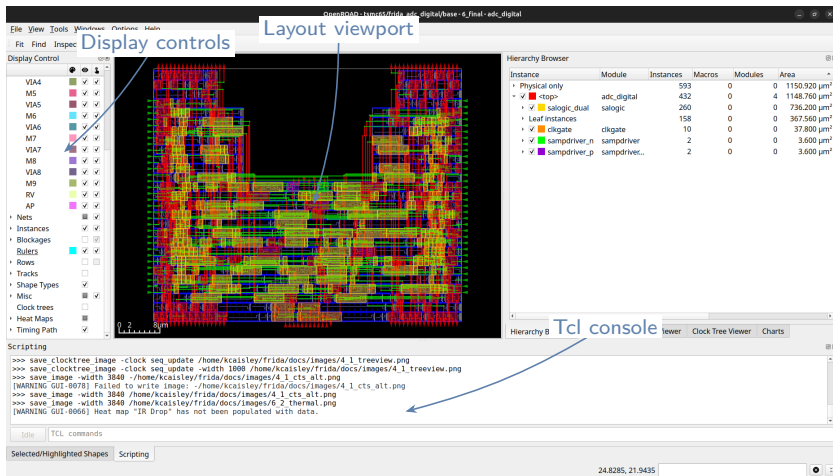
Database
GUI
Web Viewer
Synthesis
Partition Manager
Restructure
Floorplan Initialization
Pin Placement
Design for Test
Chip-level Connections
Macro Placement
Tapcell Insertion
PDN Generation
Global Placement
Static Timing Analyzer
(external)
Parasitics Estimation
Gate Resizing
Clock gating
Detailed Placement



Options

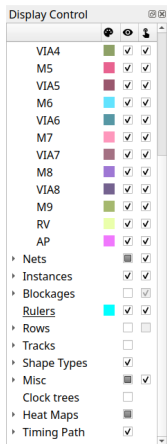
Switch Name	Description
<code>-core_area</code>	Core area coordinates in microns. Either a rectangle as <code>{llx lly urx ury}</code> , or a rectilinear polygon as a list of vertex coordinates <code>{x1 y1 x2 y2 ...}</code> . See Rectilinear Floorplans .
<code>-core_space</code>	Space around the core in microns. Allowed values are either one value for all margins or a set of four values, one for each margin. The order of the four values are: <code>{bottom top left right}</code> .
<code>-die_area</code>	Die area coordinates in microns. Either a rectangle as <code>{llx lly urx ury}</code> , or a rectilinear polygon as a list of vertex coordinates <code>{x1 y1 x2 y2 ...}</code> . See Rectilinear Floorplans .
<code>-site</code>	Site name.
<code>-utilization</code>	Percentage utilization. Allowed values are <code>double</code> in the range <code>{0-100}</code> .
<code>[-additional_sites]</code>	Tcl list of sites to make rows for (e.g. <code>{SITEXX, SITEYY}</code>)

The OpenROAD GUI



Frida ADC digital block

Display controls: what is visible



Layers and vias

Nets and instances

Rows and tracks

Visibility / selection

Hierarchy browser: what is in the design

Hierarchy Browser  

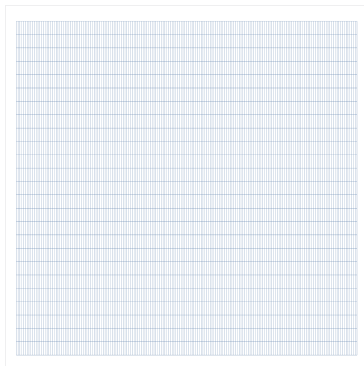
Instance	Module	Instances	Macros	Modules	Area
▸ Physical only		593	0	0	1150.920 μm^2
▾ ☒  <top>	adc_digital	432	0	4	1148.760 μm^2
▸ ☒  salogic_dual	salogic	260	0	0	736.200 μm^2
▸ Leaf instances		158	0	0	367.560 μm^2
▸ ☒  clkgate	clkgate	10	0	0	37.800 μm^2
▸ ☒  sampdriver_n	sampdriver	2	0	0	3.600 μm^2
▸ ☒  sampdriver_p	sampdriver...	2	0	0	3.600 μm^2

Instances and modules

Instance counts

Cell area

Open a saved design



```
openroad -gui -db results/floorplanning/01_floorplan.odb
```

```
Generate checkpoints with examples/floorplanning/floorplan.tcl
```

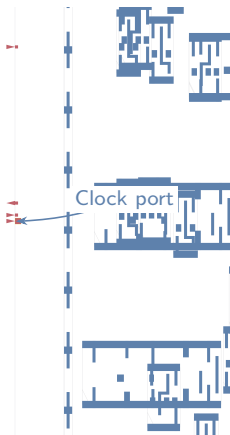
Control the view



```
gui::set_display_controls "Rows/*" visible true
```

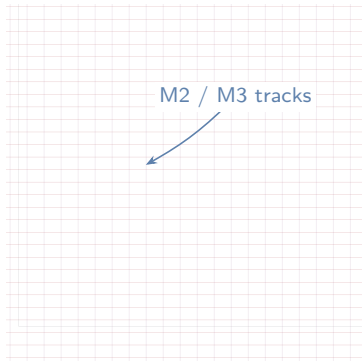
```
gui::fit
```

Inspect a pin



```
select -type BTerm -name clk
```

Export a sharp viewport



$\{x_0\ y_0\ x_1\ y_1\}$ in μm

```
save_image -width 2400 -area {1 1 5 5} tracks.png
```

Start the workshop script

```
export OPENROAD_ROOT=/path/to/OpenROAD  
openroad -gui flow.tcl
```

Run from the public repository root.

Read the physical library

```
read_lef $tech/Nangate45_tech.lef
read_lef $tech/Nangate45_stdcell.lef
Technology    Cell masters
```

Read the timing library

```
read_liberty $tech/Nangate45_typ.lib  
1.1 V      25 °C
```

Read and link the netlist

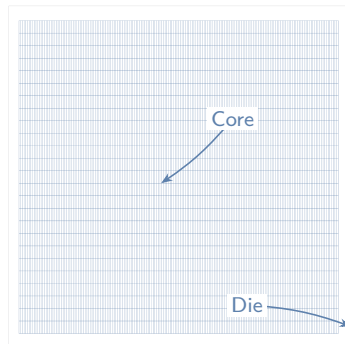
```
read_verilog $netlist
```

```
link_design gcd
```

Top module: gcd

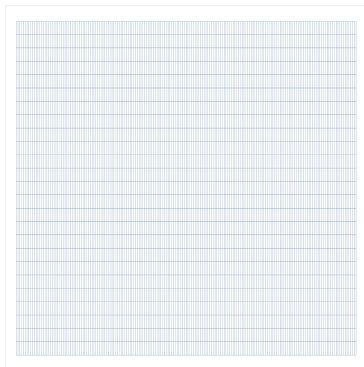
Floorplan comes next

Initialize the floorplan

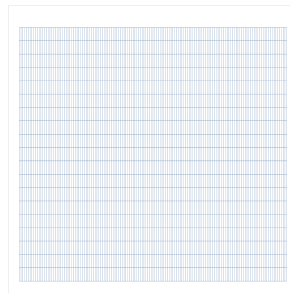


```
initialize_floorplan -site $site -utilization 40 -aspect_ratio 1 -core_space 1
```

Utilization sets the available area



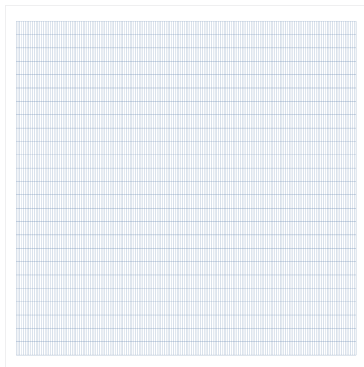
-utilization 40



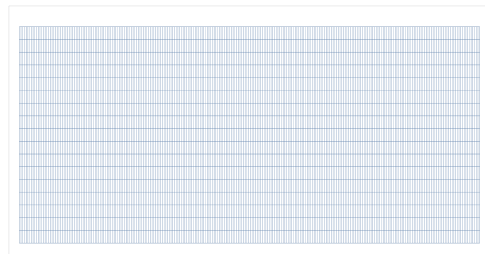
-utilization 65

$$A_{\text{core}} \approx A_{\text{cells}} / (u/100)$$

Choose the shape



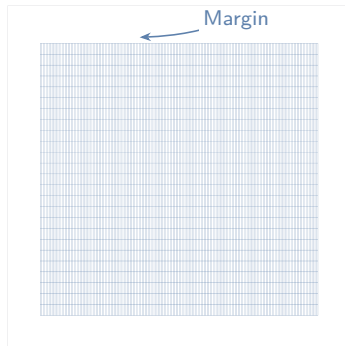
`-aspect_ratio 1`



`-aspect_ratio 0.5`

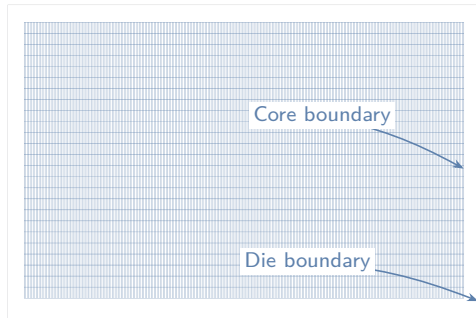
$$r = H/W \quad W = \sqrt{A/r} \quad H = \sqrt{Ar}$$

Reserve the core margin



```
initialize_floorplan -site $site -utilization 40 -core_space 4
```

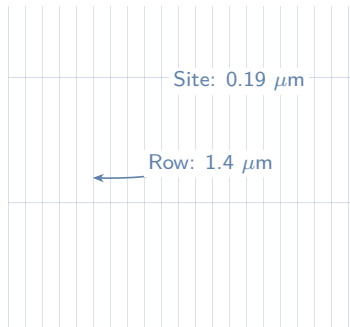
Use a prescribed footprint



Coordinates in μm

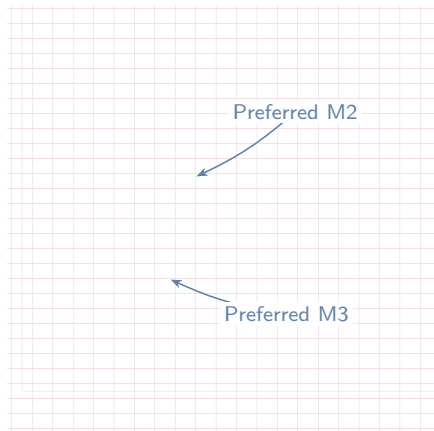
```
initialize_floorplan -site $site -die_area {0 0 60 40} -core_area {2 2 58 38}
```

Rows and sites



```
initialize_floorplan -site $site -utilization 40 -core_space 1
```

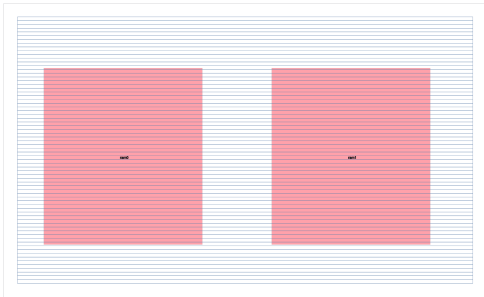
Create routing tracks



`make_tracks`

Pitch and offset from technology LEF

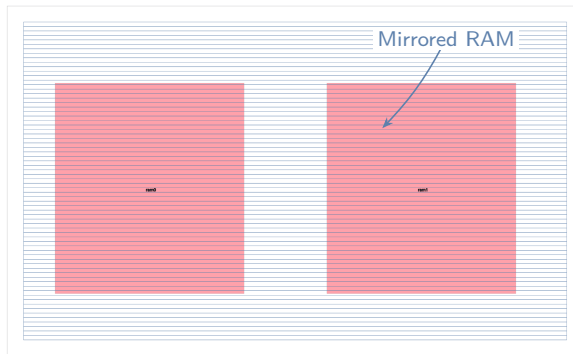
A separate macro example



Fresh session: two RAM abstracts + GCD

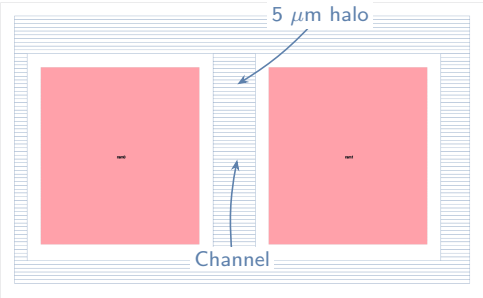
```
openroad -gui examples/floorplanning/macros.tcl
```

Place and orient a macro



```
place_macro -macro_name ram1 -location {100 20} -orientation MY
```

Leave access around the macros



```
cut_rows -halo_width_x 5 -halo_width_y 5
```

Automatic macro placement



`rtl_macro_placer`

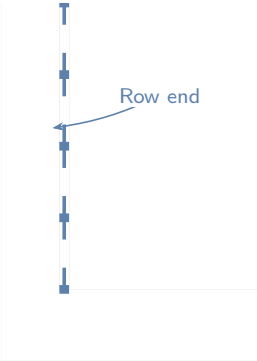
`macros_auto.tcl`: 5 μm halos

Hierarchical clustering

Simulated annealing

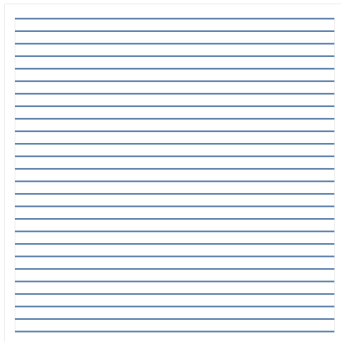
Wirelength + outline + packing

Insert taps and endcaps



```
tapcell -distance 120 -tapcell_master TAPCELL_X1 -endcap_master TAPCELL_X1
```

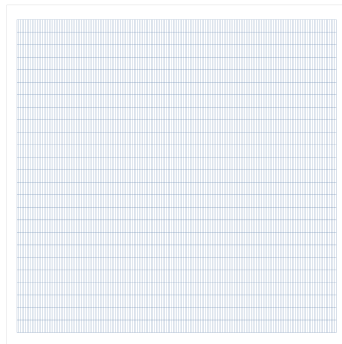
Connect the supply pins



Repeat for VSS with -ground

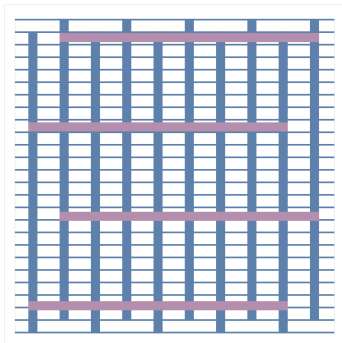
```
add_global_connection -net VDD -pin_pattern {^VDD$} -power
```

Define the power domain



```
set_voltage_domain -name CORE -power VDD -ground VSS
```

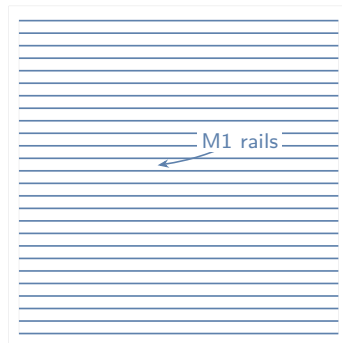
Define the grid



Grid policy; geometry at pdngen

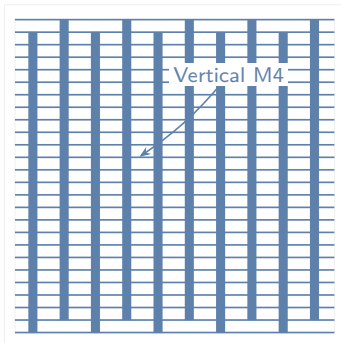
```
define_pdn_grid -name core_grid -voltage_domains CORE
```

Follow the row power rails



```
add_pdn_stripe -grid core_grid -layer metal1 -followpins
```

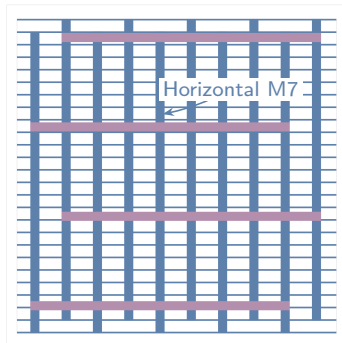
Choose strap width and pitch



Pitch repeats a VDD/VSS pair

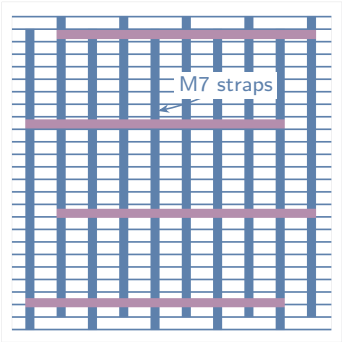
```
add_pdn_stripe -grid core_grid -layer metal4 -width 1 -pitch 7 -offset 2
```

Add a crossing strap layer



```
add_pdn_stripe -layer metal7 -width 1 -pitch 20 -offset 3 -number_of_straps 2
```

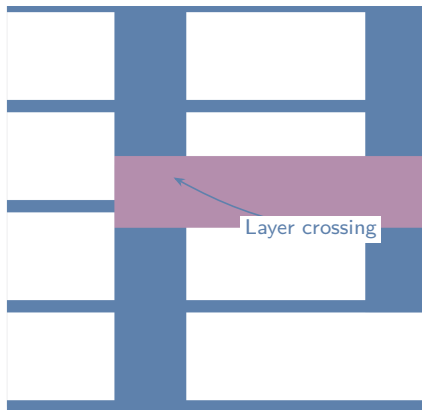
Connect the power layers



Also connect M4 to M7

```
add_pdn_connect -grid core_grid -layers {metal1 metal4}
```

Generate the power grid



pdngen

Rule-based straps and via insertion

When do we place the pins?

Pins first

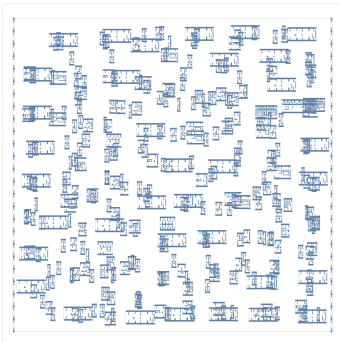
Pin constraints → pin locations → cell placement

Wirelength-driven pin locations

Rough cells → optimize pins → refine cells

Unplaced cell endpoints are modeled at the die center.

Optional: rough cells before pin optimization



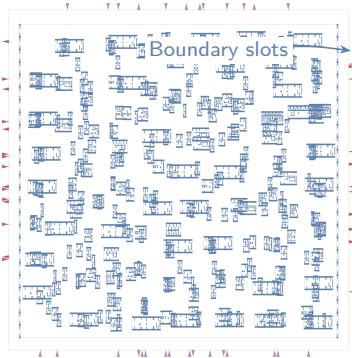
Rough global placement; detailed placement comes later

Smooth wirelength + electrostatic density

Nesterov optimization

```
global_placement -density 0.80 -pad_left 2 -pad_right 2 -skip_io
```

Automatic pin placement

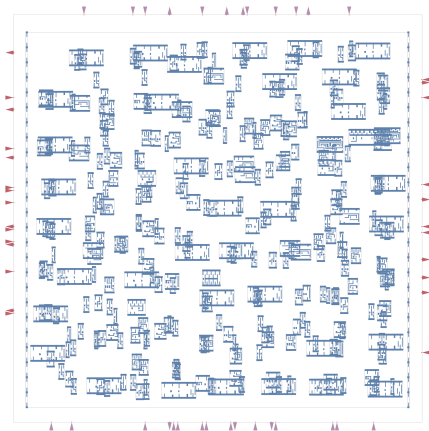


Fixed cell locations + track slots

Unplaced cells: modeled at die center

```
place_pins -hor_layers metal3 -ver_layers metal2
```

The pin-assignment objective



$$c_{ps} = \text{HPWL}(S_p \cup \{s\})$$

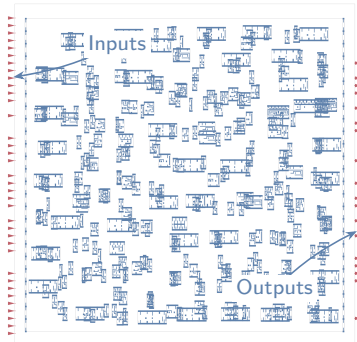
S_p : fixed cell-pin locations

Hungarian matching within slot sections

Optional simulated annealing

Geometric cost; no routed paths

Restrict pins to an edge

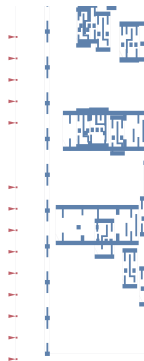


Outputs: `-region right:*`

Then rerun `place_pins`

```
set_io_pin_constraint -direction input -region left:*
```

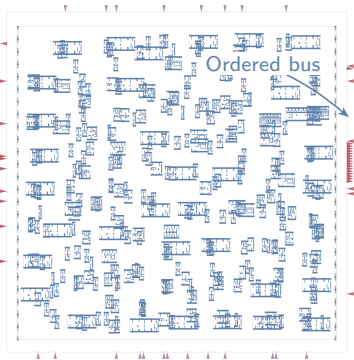
Control the minimum separation



0.8 μ m minimum; track snapped
-min_distance_in_tracks: track count

```
place_pins -hor_layers metal3 -ver_layers metal2 -min_distance 0.8
```

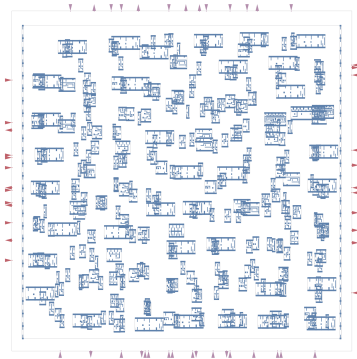
Keep a bus together



Then rerun place_pins

```
set_io_pin_constraint -pin_names {resp_msg[*]} -region right:* -group -order
```

Try the annealing solver

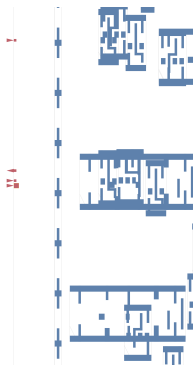


Same HPWL objective

Try edge + bus constraints

```
place_pins -hor_layers metal3 -ver_layers metal2 -annealing
```

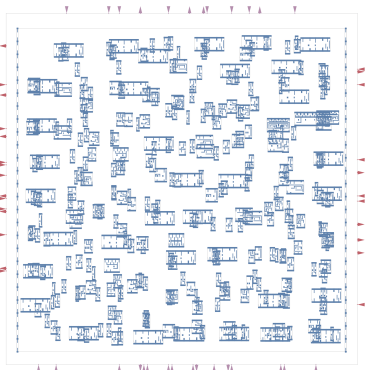
Fix one pin first



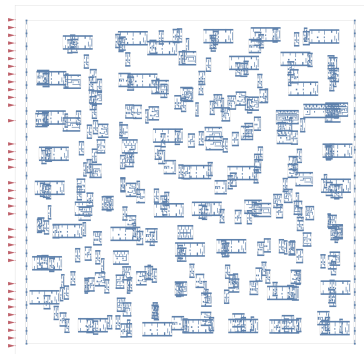
Then `place_pins` for the remaining ports

```
place_pin -pin_name clk -layer metal3 -location {0 10} -force_to_die_boundary
```

Measure the cost of constraints



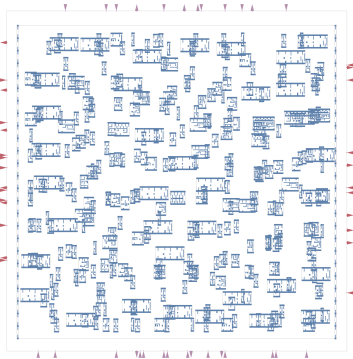
Automatic: 536.82 μm



Constrained: 1126.27 μm

Same fixed cells; constraints: opposite edges + 0.8 μm spacing

Refine cells with the pins included



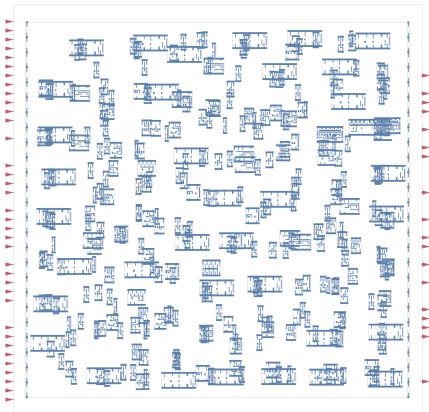
Pins now contribute to the placement objective

RUDY congestion estimate + cell inflation

-routability_use_grt: FastRoute feedback

```
global_placement -routability_driven -density 0.80 -pad_left 2 -pad_right 2
```

Save a checkpoint



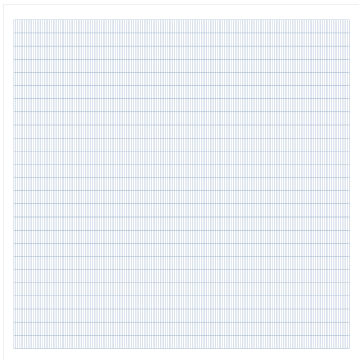
```
write_db results/floorplanning/pins.odb
```

```
write_def results/floorplanning/pins.def
```

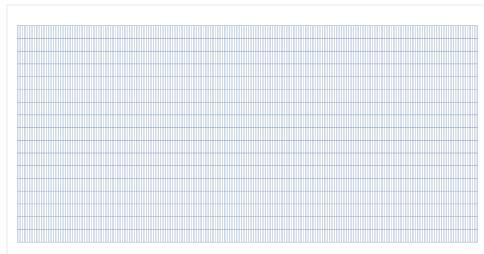
ODB: physical database

Timing restart also needs Liberty + SDC

Exercise: compare two floorplans



$$u = 40\%, H/W = 1$$



$$u = 40\%, H/W = 0.5$$

Compare I/O HPWL and pin constraints after placement